

# THE CRC ROBOTICS PROGRAMMING COMPETITION

# PRELIMINARY PROBLEM #1




A program of

AEST EAST

Version 1.0

## A FEW NOTES

- The complete rules are in section 4 of the rulebook.
- You have until **Sunday, November 17th, 11:59 pm** to submit your code.
- Feel free to use the programming forum on the CRC discord to ask questions and discuss the problem with other teams. It is there for that purpose!
- **We are giving you quick and easy-to-use template files for your code and the tests. You are required to use them.**

## USING THE TEMPLATE FILE

- The template tests call the function to test, take the output and allow you to quickly check if your code works as intended. **All your code, except additional functions you create, should be written in the function of the part of the problem you are solving.**
- Points given in the document are indications of how difficult the section is and how many points you will get if you complete it. This preliminary problem is going to be 2% of the main challenge towards the global score of the programming competition and for more points related information consult the rulebook.

## STRUCTURE

Every problem contains a small introduction like this about the basics of the problem and what is required to solve it.

### Input and output specification:

Contains the inputs and their format, and which outputs the code is required to produce and in what format they shall be.

### Sample input and output:

Contains a sample input, sometimes containing sub examples in the sample input, and what your program should return as an output.

### Explanation of the first output:

Explains briefly the logic that was used to reach the first output, given the first input.

# Advanced TakTiks

You and your team are in the final round of the TakTik 2025 competition, and your copilot wants to transmit some key TakTiks for the game. To avoid leaking information to your competitors, you decide to send encrypted messages. Since the pilot doesn't have time to decode these messages and still focus on the game, you decide to create an automatic decryption system! You need to automate every step of the message decryption process and put them together. Then, you'll have an advantage over all the other teams in the competition and increase your chances of winning TakTik 2025!

## Part 1: The start of the alphabet (20 points)

To decrypt each message, you need to select one letter per sentence. You'll need to find the letter that comes first in the alphabet for each sentence. For your secret code, uppercase letters have higher priority than lowercase letters. So, the priority for each sentence should be: A, B, C, [...], Z, a, b, c, [...], z.

**Note:** Keep in mind that the same letter has different **ASCII** values depending on whether it's uppercase or lowercase!

### Input and output specification:

You will receive as input some text in a *string* variable, where sentences are separated by end of sentence punctuation marks (. ! ?). You need to output an *array* of *strings* containing the lowest-valued letter of every sentence, in the same order of appearance as the sentences.

### Sample input:

"my c**a**t always finds a way to sneak inside! **E**very bird loves flying under the bright sky. he enjoys **c**ycling with his dog every morning."

"Samuel veut écrire du code en **P**ython. ton script est trop **c**ool! utilise-t-il **d**es modules externes?"

"**W**ait 1 second, he's not here anymore. j**e** ne le vois plus! me neither, where **d**id he go? je pense qu'il s'est perdu; cette ville est tellement **b**elle."

### Sample output:

```
['a', 'E', 'c']
['P', 'c', 'd']
['W', 'e', 'd', 'b']
```

## Explanation of the first output:

Notice that there are 3 sentences in the first input.

- In the first sentence, we look at each letter to find the one with the smallest value. We start with 'm', followed by 'y' which we won't take since it has a larger value than 'm'.
- The next letter is 'c', which has the smallest value so far, so we keep 'c' as the smallest one for now.
- Then, we come across an 'a', which we keep because it has an even lower value than 'c'.
- We can look at each letter until we reach the period '.' before confirming that the letter 'a' does in fact have the smallest value in this entire sentence.

- In the second sentence, we first see the letters 'E', 'v', 'e', 'r', 'y', and 'b'. **'E' and 'e' do not have the same ASCII value!**
- The letter 'E' has the lowest **ASCII** value here, so we take the 'E'.
- Then, we check the remaining letters in this sentence (up to the exclamation point '!') and we don't find any letter with a value smaller than 'E'.

- In the third sentence, after analyzing all the letters in a similar fashion, we see that the letter 'c' is the lowest-valued.

For the first string, we return an *array* containing 3 values of type *string*: the letter with the lowest **ASCII** value for each sentence. This gives us our final output of `['a', 'E', 'c']`.

# Part 2: Vowels and consonants (20 points)

You want to make your message even safer, and you decide to add a hidden calculation system in each sentence. You want to count the number of vowels and consonants in each sentence that you receive: you then multiply these two numbers to get the numerical value of the sentence. The letters (a,e,i,o,u,y) will be considered as vowels. Keep in mind that accented letters should also be considered in the calculation (e.g. è, ê and è count as vowels)!

## Input and output specification:

For input, you will receive a *string* containing several sentences for which you need to find the associated value (as described above).
For output, you need to output a list of integers (*int*), which are the values associated with each sentence.

## Sample input:

“My cat always finds a way to sneak inside! Every bird loves flying under the bright sky. Fred enjoys cycling with his dog every morning.”
“Pourquoi est-ce que j’ai le réflexe de regarder mes mains lorsque j'échappe un frisbee?”
“Maybe the elevator will arrive faster if I keep pressing the button. Je me suis lavé les mains après avoir cuisiné, mais elles sentent encore l’ail.”

## Sample output:

```
[270, 312, 350]
[1216]
[759, 986]
```

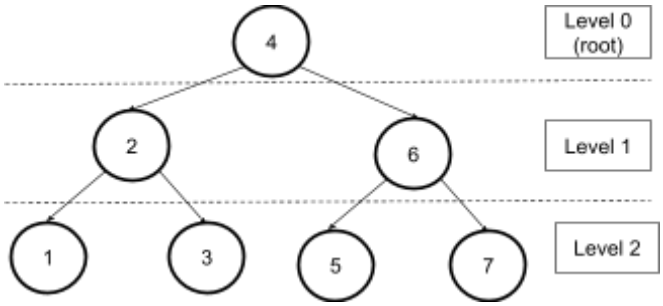
## Explanation of the first output:

In the first sentence “My cat always finds a way to sneak inside!”, we have 15 vowels and 18 consonants. We multiply these two numbers to get 270, which is the value associated with the first sentence. For the second sentence, we have 13 vowels and 24 consonants, so we find the associated value of 312. For the third sentence, there are 14 vowels and 25 consonants, which gives our associated value of 350.

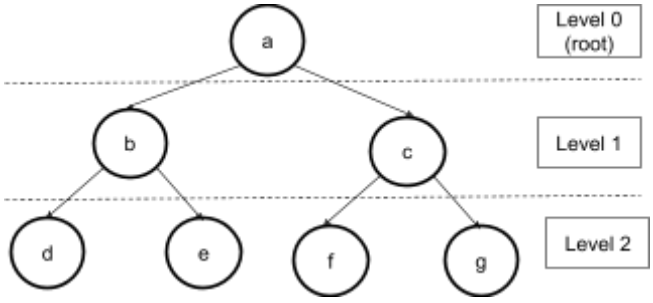
## Part 3: Trees (40 points)

In programming, trees are very useful data structures for organizing information. In a tree, the topmost element occupies level 0, and this element is called the root of the tree. We can place level 1 elements under the root which is at level 0. Every new element needs to be connected to a parent element on the level above. In our context, we'll use a tree where each element has 2 branches extending from it to its child elements on the next level, and we will only need levels 0 (the root), 1, and 2.

For this problem, since there are 3 levels (0, 1, and 2) and each one is complete, we'll have 7 values in total. More specifically, level 0 only contains the root, level 1 has two elements, and each level 1 element branches off into two level 2 elements. This gives us a total of 7 elements. In a tree, the values are organized from smallest to largest and we consider them from left to right (that is, a value on the left should be smaller than a value on the right). Here's a tree with the numbers 1 to 7 as its elements:




In the context of our decryption task, we want to take a number and place its corresponding letter in the tree. This will give our tree a more interesting look! We'll position the letters in the tree according to the position of their associated number. Here's an example with the numbers [1, 2, 3, 4, 5, 6, 7] and their corresponding letters ['d', 'b', 'e', 'a', 'f', 'c', 'g']. In this example, 'd' is associated with 1, 'b' is associated with 2, and so on until we reach 'g' which is associated with 7. We substitute the values 1-7 from our previous example above with their associated letters to get a new tree:




There are several ways to traverse (i.e. go through all the elements) of a tree, but for our decryption purposes, we'll look through a tree level by level, which is an algorithm called Breadth-First Search (BFS). So, we'll traverse one level after the other and read

values from left to right. If we traverse the tree from the example just above, we obtain the result:
[‘a’, ‘b’, ‘c’, ‘d’, ‘e’, ‘f’, ‘g’].

To summarize, we’ll construct a tree with the list of 7 numbers received as input, replace each number with their corresponding letter, and finally, traverse the tree using breadth-first search.

To learn more about trees in general, here are two interesting resources:
https://www.freecodecamp.org/news/all-you-need-to-know-about-tree-data-structures-bceacb85490c/
https://www.geeksforgeeks.org/introduction-to-tree-data-structure/

## Input and output specification:

For input, you will receive two lists (*arrays*), each containing 7 elements. The first list will contain the numbers to place in the tree, and the second list will be the letters associated with each number.
For output, you will give the 7 letters in an *array*, in the order given by breadth-first search.

## Sample input:

[31, 44, 14, 33, 43, 26, 57], ['i', 'o', 'n', 'u', 'c', 's', 'e']

[2, 4, 6, 5, 3, 1, 7], ['b', 'a', 'c', 'f', 'e', 'd', 'g']

[14, 41, 10, 59, 99, 109, 90], ['u', 'e', 'l', 'x', 'p', 't', 'f']

## Sample output:

['u', 's', 'o', 'n', 'i', 'c', 'e']

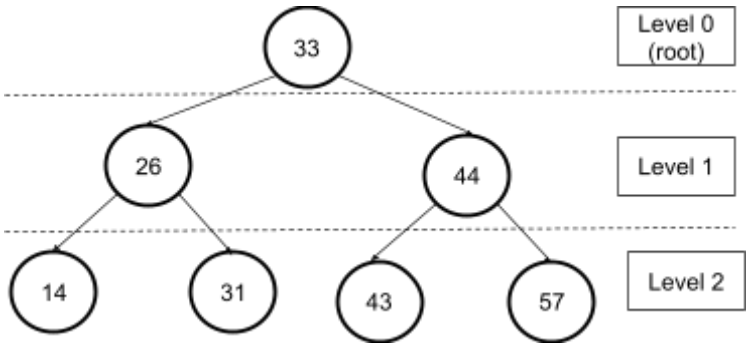
['a', 'b', 'c', 'd', 'e', 'f', 'g']

['x', 'u', 'p', 'l', 'e', 'f', 't']

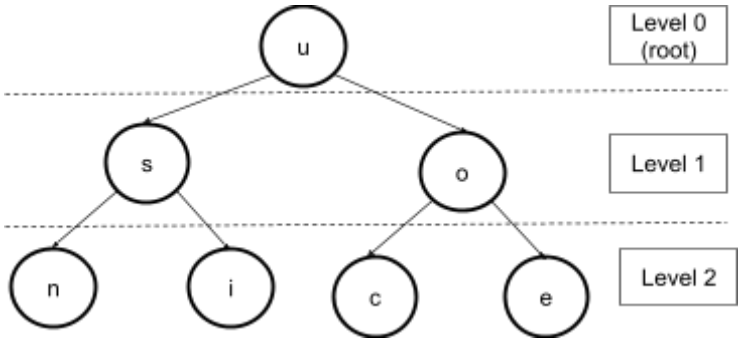
## Explanation of the first output:

In the first example, we have these numbers and corresponding letters:
[31, 44, 14, 33, 43, 26, 57]
['i', 'o', 'n', 'u', 'c', 's', 'e']

We’ll start by placing the numbers in the right places in our tree. That means we need to make sure the values go from smallest to largest, from left to right. Here’s the tree once we’ve placed the numbers:




Now we can replace each number with their corresponding letter. This gives us the following tree:




Once the tree is populated with the letters, we can traverse it using breadth-first search and obtain our final result:
['u', 's', 'o', 'n', 'i', 'c', 'e']

## Part 4: Draw the trees (15 points)

Now that you're able to organize elements of an *array* in the form of a tree, you'll need to find a way to display the trees! You will still receive an *array* (i.e. a list) containing 7 values. From this list, you'll need to display the elements on several lines to form a two-dimensional visual representation of the tree! The list elements will be placed in the same order as the *array* outputted in part 3: that is, level by level (breadth-first search). The visual representation of a tree (the tree that you display) must be made up of the list elements, of spaces (whitespace) [**ASCII** code 32], and of arrows (↘ and ↙) [unicode 2198 and unicode 2199]. To help you display your trees correctly, we'll provide you with an example with the values 1-7. You can use this tree to display your final outputs.

### Input and output specification:

For input, you will receive an array of letters that needs to be displayed as a tree.
For output, you need to provide an array of strings that forms a visual representation of the tree.

### Sample input:

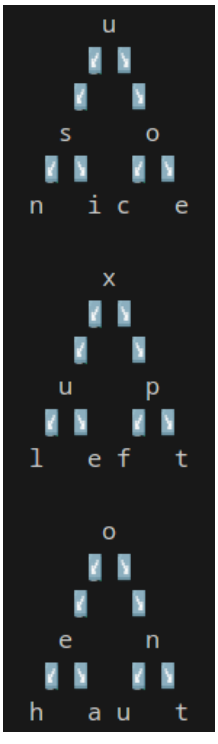
['u', 's', 'o', 'n', 'i', 'c', 'e']
['x', 'u', 'f', 'l', 'e', 'p', 't']
['o', 'e', 'u', 'h', 'n', 'a', 't']

### Sample output:

* The formatting in Google Docs is not the same as the formatting in the console

```
[
"      u       ",
"    ↙ ↘       ",
"   ↙   ↘      ",
"  s     o     ",
" ↙ ↘   ↙ ↘    ",
"n   i c   e"]

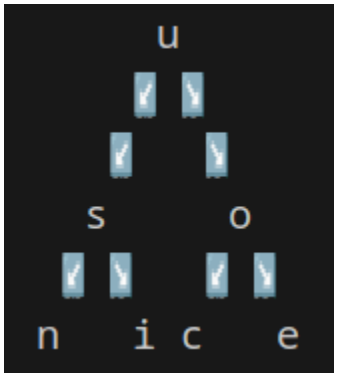
[
"      x       ",
"    ↙ ↘       ",
"   ↙   ↘      ",
"  u     p     ",
" ↙ ↘   ↙ ↘    ",
"l   e f   t"]
```



```
[
"      o       ",
"    ↙ ↘       ",
"   ↙   ↘      ",
"  e     n     ",
" ↙ ↘   ↙ ↘    ",
"h   a u   t"]
```

## Explanation of the first output:

In the first example, our input is the *array* of letters [‘u’, ‘s’, ‘o’, ‘n’, ‘i’, ‘c’, ‘e’]. We can place each value in the tree by using breadth-first search or by replacing the values of the provided example. Once the values are replaced, we can see the result by using the helper function print_tree() which displays the following:




We can return the final *array* of strings with the replaced values, so we return:

```
[
"      u       ",
"    ↙ ↘       ",
"   ↙   ↘      ",
"  s     o     ",
" ↙ ↘   ↙ ↘    ",
"n   i c   e"]
```

The formatting in Google Docs differs from that of the console, so be sure to use the provided example to help you display and format your output correctly and to pass the tests!

## Part 5: Decode the secret messages (15 points)

In this part you need to combine all the previous parts to decode the encrypted messages. You will receive a *string* containing 7 sentences. For each sentence, you will need to find the letters with the smallest **ASCII** values (like in part 1) and the product of the number of vowels and consonants (like in part 2). You will then have to organize the sentence as a tree and display it using your work from parts 3 and 4. This way, you will be able to decode the full secret message that you can show your pilot during the game. This is the last step to give your team a major advantage during TakTik 2025!

### Input and output specification:

As input, you will receive a *string* containing 7 sentences. These 7 sentences are the one you will decrypt to find the hidden message.
As output, you will provide an *array* of *strings* that represents the tree representation of the secret message you've decoded.

### Sample input:

"**T**ommy! **A**pproche! **I**ci ils font des gros robots! **T**hey are for the TakTik 2025 competition. **K**ooooolll robots, that seems like so much fun! **K**evin might also be interested to join the robotics team with us. **S**weet, if both of you are joining, we will have a super good team and might win the event!"

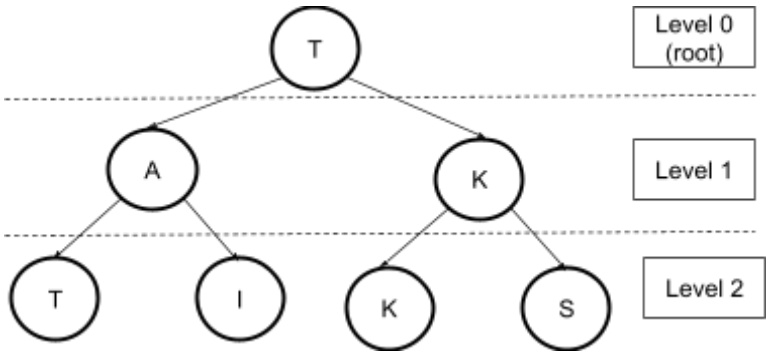
### Sample output:

```
[
"      T       ",
"    ↙ ↘       ",
"   ↙   ↘     ",
"  A     K    ",
" ↙ ↘   ↙ ↘  ",
"T   I K   S"
]
```




### Explanation of the first output:

- In this problem, we take 7 sentences as input. We start by finding the 7 associated letters according to the rules from part 1. We then obtain the following letters: ['T', 'A', 'I', 'T', 'K', 'K', 'S']
- Next, we take the product of the number of consonants and vowels for each sentence to get these values: [6, 15, 120, 221, 330, 672, 1170].
- Once we have this information, we can construct a tree using the values we obtained in the first two parts:




- Using *breadth-first search* (BFS), we can retrieve the letters: ['T', 'A', 'K', 'T', 'I', 'K', 'S']
- Once we have them in the correct order for our final message, we can format everything to display it nicely and obtain the following:

```
[
"      T       ",
"    ↙ ↘       ",
"   ↙   ↘      ",
"  A     K     ",
" ↙ ↘   ↙ ↘    ",
"T   I K   S"]
```

This gives us the following tree representation in the console:

